Diseño del reemplazo de Caja Negra

Proponemos un servicio HTTP determinista que reproduce el comportamiento observable de Caja Negra, incluidos errores y memoria entre requests. La unidad de diseño es una transición de estado: una misma entrada puede responder distinto según la secuencia anterior. La fidelidad al protocolo gobierna las decisiones de implementación.

Reto 2 de la hackatón de IA | Entregable 1 de 2 | Diseño propuesto | 10 de octubre de 2026

Objetivo y criterio de éxito

El evaluador debe recibir el mismo status HTTP y el mismo cuerpo en cada paso. Para JSON se ignora solo el orden de claves de objetos; se conservan tipos, valores, orden de arrays, mayúsculas y espacios de strings. Si alguno de los cuerpos no es JSON, se compara el texto exacto. Un test falla cuando falla cualquiera de sus pasos. Los headers no se comparan y cada paso tiene un límite de 5 segundos [R1].

Alcance y límites

Incluimos POST /msg, POST /__reset, GET /health, autenticación, rutas y métodos no admitidos, validaciones, comandos, contadores de identificadores y efectos acumulados. Conservamos nombres inconsistentes de campos, errores ortográficos y respuestas de error. El servicio nuevo será autónomo: no podrá consultar al original ni a otro reemplazo durante la evaluación [R1].

Dejamos fuera del reto alta disponibilidad, réplicas, una base de datos durable, interfaces de usuario y mejoras del contrato. El set se comunica con un solo proceso y reinicia el estado antes de cada test; esas capacidades añadirían coordinación y riesgo sin resolver la equivalencia. La exploración y el log sí se guardan de forma durable para reconstruir decisiones.

Restricciones que cambian el diseño

Restricción del reto [R1]	Consecuencia de diseño
40 tests de un paso, 45 de 2 a 5 y 15 de 6 a 12	Priorizar secuencias y transiciones; aprobar ejemplos sueltos no basta.
Ningún test repite los 20 ejemplos	Inferir reglas y verificar entradas nuevas; no memorizar respuestas.
10 requests/s y 8.000/día por token	Sondas secuenciales con cuota, hipótesis y registro por request.
Versión vigente y corte a las 20:00	Versionar evidencia y repetir casos centinela antes del cierre.

Línea base de exploración

Se enviaron 76 requests en la ventana 10 de octubre de 2026, 11:15:43 a 11:16:56 (America/Bogota). Dos replays independientes después de reset coincidieron con los 20 ejemplos: 20/20 cada uno, 40/40 respuestas en total (log 12 a 53). Ninguna respuesta llevó X-Frontier-Infra: rate-limit. Esta medición valida la referencia actual contra los ejemplos; todavía no mide un reemplazo.

Fuentes de contexto: [R1] README.md; [R2] pruebas/ejemplos.jsonl; [E] experimentos.jsonl. La línea base orienta las decisiones de diseño y el plan de medición. El catálogo detallado de comportamientos se documentará en la etapa de especificación.

Arquitectura y decisiones justificadas

La arquitectura propuesta separa exploración y ejecución. Durante el desarrollo, un recolector propio llama al original y produce evidencia. Durante la evaluación, el único camino es cliente, adaptador HTTP, reglas, motor de estado y serializador. Los diagramas del entregable 2 muestran ambas fronteras y enlazan estas decisiones.

D1 Un proceso con módulos internos

Elegimos un único proceso Node.js con un motor determinista y módulos pequeños de protocolo, comandos, estado y respuesta. Descartamos microservicios: las secuencias y los contadores exigirían coordinación remota sin beneficio para el set de un solo proceso. Costo aceptado: el proceso es un punto único de fallo; se verificará su estabilidad con secuencias repetidas.

D2 Estado en memoria con cambios atómicos

Elegimos mapas en memoria, un ámbito para el token de evaluación y ejecución serial por ámbito. Reset reemplaza el estado completo por su estado inicial. Descartamos SQL, Redis y varias réplicas: añaden persistencia y concurrencia que el reto no requiere. Costo: reiniciar el proceso pierde el estado; no se despliega ni reinicia durante un test. El token de exploración y el de evaluación se configuran por separado.

D3 Adaptador HTTP y reglas explícitas

Elegimos node:http para controlar ruta, método, cuerpo y status [T1], con un serializador único que preserve valores y texto. Descartamos validadores automáticos de esquema y normalización global: pueden cambiar los errores, su precedencia o los espacios. Costo: implementamos y probamos cada borde HTTP. El TLS público termina en la plataforma de despliegue, cuya fidelidad de rutas debe comprobarse desde internet.

D4 Reglas inferidas y ejecución sin LLM

Elegimos funciones comprobables y transiciones explícitas, incluidas las excepciones observadas. Descartamos memorizar los ejemplos, usar un LLM en cada request y consultar al original: los tests son nuevos, el límite es 5 segundos y llamar al original invalida el set. Costo: descubrir la fórmula de control y los casos de estado requiere experimentos; una tabla de mensajes conocidos no es una solución general.

D5 Evidencia y reglas versionadas

Elegimos registrar cada sonda antes de usar su conclusión y vincular cada regla con IDs y confianza. Descartamos actualizar producción automáticamente al observar una diferencia: un cambio aislado puede venir de infraestructura o de estado distinto. Costo: promoción manual de una versión tras repetir la secuencia y pasar regresión. Una reversión de despliegue protege disponibilidad, pero una versión antigua puede perder fidelidad si cambió el original.

D6 Exploración controlada y medición por secuencia

Elegimos un recolector secuencial a un máximo planificado de 4 requests/s, sin reintentos invisibles, y un comparador que reinicia ambos servicios antes de cada secuencia. Descartamos fuzzing masivo sin hipótesis y retries automáticos: gastan cuota y alteran estado. Costo: menor cantidad de sondas; se priorizan por impacto, incertidumbre y capacidad de distinguir hipótesis.

T1: documentación oficial de HTTP de Node.js, https://nodejs.org/api/http.html, consultada el 10 de octubre de 2026. La elección de Node.js y de módulos es una propuesta del reemplazo; no es una deducción sobre la tecnología del original. El runtime y sus dependencias se fijarán al implementar.

Plan de pruebas y métricas de aceptación

Antes de implementar, definimos un banco de 100 secuencias nuevas con la misma mezcla del evaluador: 40 de un paso, 45 de 2 a 5 y 15 de 6 a 12. Los 20 ejemplos forman una regresión separada y no cuentan como cobertura de entradas nuevas. Para cada secuencia se guarda el resultado del original desde reset, su versión y los IDs de evidencia.

Matriz de casos que debe cubrir el banco

Familia	Casos discriminantes	Qué se compara
HTTP y autenticación	Métodos, rutas, token ausente e inválido, query y barra final; cruzar token inválido con JSON mal formado.	Status y cuerpo; orden entre rechazo de ruta, auth y cuerpo.
Cuerpo y límites	m ausente, null, número, vacío; raíz array; JSON inválido; UTF-8 y tamaños en el umbral descubierto.	Error exacto y efecto posterior sobre estado.
Integridad y parser	Control ausente, no numérico, incorrecto y correcto; ceros, signos, espacios y separadores extra.	Ramas PING y otros comandos; orden de errores.
Comandos y sugerencias	Variantes de capitalización, comandos parecidos y desconocidos; 0, 1 o varios argumentos extra.	Aridad, nombre de campos, strings y sugerencia opcional.
Cuentas y contadores	ALTA repetido, error antes de ALTA, reset entre altas, cuenta inexistente.	IDs, saldos iniciales y consumo de contadores.
Importes y transferencias	Tras hallar el umbral L: L-1, L, L+1; saldo menor, igual y mayor al importe; autoenvío; 0, negativos y decimales.	Precedencia E506/E507, coerción y saldo posterior.
Anulación y secuencias largas	Anular depósito y transferencia, repetir, ID inexistente y gastar antes de anular; incluir consultas entre efectos.	Resultado completo en cada paso, marca de anulación y saldos.
Reinicio y cambios	Reset dos veces; repetir la misma secuencia; centinelas de validación, límite y anulación.	Estado inicial y detección de diferencias entre versiones.

Métricas y puertas de aceptación propuestas

Métrica	Cálculo o medición	Objetivo
Equivalencia por secuencia	Secuencias sin ninguna diferencia / secuencias ejecutadas. Informar también diferencias por paso.	100/100 en banco local; no garantiza el set oculto.
Equivalencia por segmento	Aprobadas / ejecutadas por longitud y familia; reportar 40, 45 y 15 por separado.	100 % en cada segmento del banco.
Fidelidad de reset	Repetir 10 veces una secuencia de 12 pasos, cada vez desde reset.	10/10 resultados idénticos.
Latencia pública	En al menos 200 pasos contra el reemplazo: p95 y máximo; timeout cuenta como fallo.	p95 < 1 s y todos < 5 s.
Trazabilidad	Reglas activas con evidencia válida o supuesto explícito / reglas activas.	100 %; 0 supuestos en comportamientos críticos al cierre.

Los objetivos son criterios de diseño, no resultados obtenidos. Hoy solo están medidos los dos replays contra el original. El comparador interpreta JSON si ambos cuerpos son JSON; compara objetos sin orden de claves, arrays en orden y valores por tipo. Si alguno no es JSON, usa texto exacto. No aplica trim, tolerancias numéricas ni correcciones de strings.

Plan de descubrimiento y salida a evaluación

Resolver primero la incertidumbre que bloquea generalizar

P0 · Control e integridad. Proponer fórmulas candidatas usando los ejemplos; variar un solo carácter y usar nombres nuevos para distinguirlas. Una fórmula se promueve solo después de predecir controles válidos para al menos 10 mensajes nuevos de varios comandos y rechazar controles alterados. PING no sirve para validar la fórmula porque acepta controles no válidos en las sondas.

P0 · Memoria y errores. Construir secuencias mínimas que separen hipótesis: error seguido de operación válida para ver consumo de IDs; transferencia al límite con fondos suficientes; misma transferencia con fondos insuficientes; anulación con y sin gasto posterior. Consultar saldos antes y después. El éxito aislado no revela todos los efectos.

P1 · Protocolo completo. Medir normalización, números, nombres, rutas y sugerencias. Usar clases de equivalencia y búsquedas de umbral en vez de combinaciones indiscriminadas. Un caso con dos fallos identifica precedencia; un caso con un fallo identifica su cuerpo. Mantener ambas clases en regresión.

Presupuesto de exploración

Uso planificado	Máximo de requests del original
Reconocimiento realizado en esta etapa	76
Fórmula del control, parser y entradas nuevas	1.600
Estados, anulación y orden de errores	1.200
Banco diferencial y repeticiones	1.400
Centinelas y verificación antes del cierre	500
Reserva para cambios y casos inesperados	1.000
Total de esta propuesta	5.776 de 8.000 por día y token

El presupuesto incluye resets; verificar el consumo del token por otros trabajos. El recolector guarda hipótesis, secuencia, id, X-Req, status, cuerpo crudo y hora. Un fallo de DNS sin request enviado no cuenta como respuesta. Una respuesta rate-limit se registra y se excluye de inferencias; reiniciar la secuencia después de recuperar cuota. No guardar credenciales en documentos ni logs.

Condiciones para desplegar y entregar

Implementar HTTP y reset, parser y reglas, luego transiciones y respuestas exactas. Cada fallo diferencial se reduce a su secuencia mínima y se convierte en regresión. Un error puede cambiar estado si la evidencia lo exige; la atomicidad no implica ausencia de efectos.

Configurar una instancia sin autoscaling, token de evaluación secreto y versión fija de reglas. Health devuelve 200 al estar listo. Verificar desde internet autenticación, rutas, reset, corpus y latencia; comprobar ausencia de llamadas al original. Evitar reinicios durante una secuencia.

Antes de las 20:00 del evento, repetir centinelas y el banco completo; confirmar la zona horaria al registrar el entrypoint. Entregar URL, especificación en siete secciones, log y resultados. Al corte, los tokens de participante dejan de servir para explorar; el reemplazo debe funcionar sin ellos.

Supuestos pendientes que bloquean afirmar equivalencia general: fórmula del control, límites por comando, semántica de anulación, coerción numérica, sugerencias y precedencia completa. Si el tiempo impide probar alguno, definir el fallback más cercano a la evidencia, marcarlo supuesto y reportar los casos afectados; no inventar una afirmación confirmada. Estos dos documentos son el diseño previo a la implementación y despliegue.